

MicroServices

Study Guide

Module 2

Developing BookStoreMS –V.1.0

Author
Srinivas Dande





CourseCube[®]
(Formerly Java Learning Center)



BookPrice MicroService

- Application Name : **MyBookPriceMS**
- Ports : 9000 , 9100 ,9200

Steps to Develop MyBookPriceMS:

A)Setup the Database:

1) Create and open the database

```
drop database jlcbookpricedb;  
create database jlcbookpricedb;  
use jlcbookpricedb;
```

2) Create the tables required

```
create table mybookprice(  
book_id int primary key,  
price double,  
offer double  
);
```

3) Insert some sample records

```
insert into mybookprice values(101,2000,10);  
insert into mybookprice values(102,1000,10);  
insert into mybookprice values(103,2000,10);  
insert into mybookprice values(104,1000,10);  
insert into mybookprice values(105,3000,10);  
insert into mybookprice values(106,2000,10);  
insert into mybookprice values(107,2000,10);  
insert into mybookprice values(108,1000,10);  
insert into mybookprice values(109,3000,10);
```

4) Select and see the sample records

```
select * from mybookprice;
```



B)Setup the Boot Project

5) Create the Maven Project.

Group Id: com.jlcindia.bookprice

Artifact Id : BookPriceMS

Version : 1.0

6) Remove the following

a)src/test/java

b)src/test/resources

7) Update pom.xml

8) RUN Maven -> Update Project.

9) Create application.properties under src/main/resources

10) Update application.properties

11) Create the Package called com.jlcindia.bookprice

12) Write BootApplication Startup Class.

***BookPriceApplication.java

13) Write Spring Bean Configuration Class

*** JLCBookPriceConfig.java

C)Develop the BookPrice MicroService Code:

14) Write the JPA Entity

*** BookPrice.java

15) Write DAO Layer components

*** BookPriceDAO.java

16) Write Service Layer components

*** BookPriceService.java

*** BookPriceServiceImpl.java



17) Write Controller Layer component

*** BookPriceController.java

```
@GetMapping("/bookPrice/{bookId}")
public BookPrice getBookPrice(@PathVariable Integer bookId)

@GetMapping("/offeredPrice/{bookId}")
public double getOfferedPrice(@PathVariable Integer bookId)
```

18) Start the Application

19) Open Swagger UI with Browser

<http://localhost:9000/swagger-ui.html>

20) Test with Browser

<http://localhost:9000/bookPrice/101>
<http://localhost:9000/offeredPrice/101>

BookPriceMS: Files required

1. BookPriceController.java	2. BookPriceService.java
3. BookPriceServiceImpl.java	4. BookPriceDAO.java
5. BookPrice.java	6. JLCBookPriceConfig.java
7. StartMyBookPriceMS.java	8. application.properties
9. pom.xml	

1.BookPriceController.java

```
package com.jlcindia.bookprice;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

import io.swagger.annotations.ApiOperation;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
```



@CrossOrigin // CORS

@RestController

```
public class BookPriceController {
```

```
    static Logger log = LoggerFactory.getLogger(BookPriceController.class);
```

@Autowired

```
    BookPriceService bookPriceService;
```

@GetMapping("/bookPrice/{bookId}")

@ApiOperation(value = " getBookPrice", response = BookPrice.class, notes = "Gives BookPrice Object of given bookId")

```
    public BookPrice getBookPrice(@PathVariable Integer bookId) {  
        log.info("---BookPriceController---getBookPrice()-----");
```

```
        BookPrice bookPrice = bookPriceService.getBookPriceById(bookId);  
        return bookPrice;
```

```
    }
```

@GetMapping("/offeredPrice/{bookId}")

@ApiOperation(value = " getOfferedPrice", response = double.class, notes = "Gives Offered Price of given bookId")

```
    public double getOfferedPrice(@PathVariable Integer bookId) {  
        log.info("---BookPriceController---getOfferedPrice()-----");
```

```
        double offeredPrice = bookPriceService.getOfferedPriceById(bookId);  
        return offeredPrice;
```

```
    }
```

```
}
```

2.BookPriceService.java

```
package com.jlcindia.bookprice;
```

```
/*
```

```
 * @Author : Srinivas Dande
```

```
 * @company : Java Learning Center
```

```
 */
```

```
public interface BookPriceService {
```

```
    public BookPrice getBookPriceById(Integer bookId);
```

```
    public double getOfferedPriceById(Integer bookId);
```

```
}
```



3. BookPriceServiceImpl.java

```
package com.jlcindia.bookprice;

import org.springframework.transaction.annotation.Transactional;
import org.slf4j.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@Service
@Transactional
public class BookPriceServiceImpl implements BookPriceService{

    static Logger log=LoggerFactory.getLogger(BookPriceServiceImpl.class);

    @Autowired
    BookPriceDAO bookPriceDAO;

    @Override
    public double getOfferedPriceById(Integer bookId) {

        log.info("---BookPriceServiceImpl---getOfferedPriceById()-----");
        double offerPrice=0.0;
        Optional<BookPrice> opt=bookPriceDAO.findById(bookId);
        if(opt.isPresent()) {
            BookPrice bookPrice=opt.get();
            double price=bookPrice.getPrice();
            double offer=bookPrice.getOffer();

            if(offer<=0) {
                return price;
            }
            offerPrice = price - price * offer/100;
        }
        return offerPrice;
    }

    @Override
    public BookPrice getBookPriceById(Integer bookId) {
        log.info("---BookPriceServiceImpl---getBookPriceById()-----");
```



```
        BookPrice bookPrice=null;
        Optional<BookPrice> opt=bookPriceDAO.findById(bookId);
        if(opt.isPresent()) {
            bookPrice=opt.get();
        }
        return bookPrice;
    }
}
```

4.BookPriceDAO.java

```
package com.jlcindia.bookprice;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@Repository
public interface BookPriceDAO extends JpaRepository<BookPrice,Integer> {

}
```

5.BookPrice.java

```
package com.jlcindia.bookprice;

import javax.persistence.*;

import io.swagger.annotations.ApiModel;
import io.swagger.annotations.ApiModelProperty;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@Entity
@Table(name = "mybookprice", schema = "jlcbookpricedb")
@ApiModel("BookPrice which contains BookId, price and offer/discount")
public class BookPrice {
```



```
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "bookId_generator")
@SequenceGenerator(name = "bookId_generator", sequenceName = "mybookId_gen",
initialValue = 110, allocationSize = 1)
@Column(name = "book_id")
@ApiModelProperty("holds BookId")
private Integer bookId;

@Column(name = "price")
@ApiModelProperty("holds price")
private double price;

@Column(name = "offer")
@ApiModelProperty("holds offer/discount")
private double offer;

//Constructors
//Setters and Getters

}
```

6.JLCBookPriceConfig.java

```
package com.jlcindia.bookprice;

import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
import org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

import com.google.common.base.Predicates;

import springfox.documentation.builders.PathSelectors;
import springfox.documentation.builders.RequestHandlerSelectors;
import springfox.documentation.service.ApiInfo;
import springfox.documentation.service.Contact;
import springfox.documentation.spi.DocumentationType;
import springfox.documentation.spring.web.plugins.Docket;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
public class JLCBookPriceConfig implements WebMvcConfigurer {
```



```
private ApiInfo getApiDetails() {
return new ApiInfo("JLC BookStore- API",
    "BookPrice API - part of BookStore", "1.0",
    "Free to use ",
    new Contact("Srinivas Dande", "https://www.jlcindia.com", "sri@jlcindia.com"),
    "API Under Free Licence",
    "https://www.jlcindia.com");
}

@Bean
public Docket api() {

return new Docket(DocumentationType.SWAGGER_2).select().paths(PathSelectors.any())
.apis(Predicates.not(RequestHandlerSelectors.basePackage("org.springframework.boot")))
.build().apiInfo(getApiDetails());

}

@Override
public void addResourceHandlers(ResourceHandlerRegistry registry) {
//enabling swagger-ui
registry.addResourceHandler("swagger-ui.html").addResourceLocations("classpath:/META-INF/resources/");

registry.addResourceHandler("/webjars/**").addResourceLocations("classpath:/META-INF/resources/webjars/");
}
}
```

7. BookPriceApplication.java

```
package com.jlcindia.bookprice;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

import springfox.documentation.swagger2.annotations.EnableSwagger2;
/*
* @Author : Srinivas Dande
* @company : Java Learning Center
**/
```



@SpringBootApplication

@EnableSwagger2

```
public class BookPriceApplication {
```

```
    static Logger log=LoggerFactory.getLogger(BookPriceApplication.class);
```

```
    public static void main(String args[]) {
```

```
        log.info(" BookPriceMS - Begin ");
```

```
        SpringApplication.run(BookPriceApplication.class, args);
```

```
        log.info(" BookPriceMS - End ");
```

```
    }
```

```
}
```

8. application.properties

spring.application.name=**MyBookPriceMS**

server.port=**9000**

spring.datasource.driver-class-name=com.mysql.jdbc.Driver

spring.datasource.url=jdbc:mysql://localhost:3306/**jlcbookpricedb**

spring.datasource.username=root

spring.datasource.password=srinivas

spring.datasource.hikari.connectionTimeout=20000

spring.datasource.hikari.maximumPoolSize=5

logging.level.root=INFO

logging.pattern.console=%-5level %logger{36} - %msg%n

spring.jpa.show-sql=true

spring.jpa.hibernate.ddl-auto=update

9. pom.xml

```
<project ...>
```

```
<modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.jlcindia.bookprice</groupId>
```

```
<artifactId>BookPriceMS</artifactId>
```

```
<version>1.0</version>
```

```
<packaging>jar</packaging>
```

```
<name>MyBookPriceMS</name>
```

```
<url>https://www.jlcindia.com</url>
```



```
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.5.3</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>

<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger2</artifactId>
<version>2.4.0</version>
</dependency>

<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger-ui</artifactId>
<version>2.4.0</version>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-devtools</artifactId>
</dependency>

</dependencies>
```



```
<build>
<finalName>MyBookPriceMS</finalName>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>

</project>
```

MicroService Name	Rest Endpoints
MyBookPriceMS	http://localhost:9000/bookPrice/{bookId}
MyBookPriceMS	http://localhost:9000/offeredPrice/{bookId}